模型预训练：

通过大量数据预先训练出一个通用神经网络，它学习了大量通用的特征。

浅层学习了**边缘、颜色、纹理**等低级特征；深层学习了物体**部位、种类、风格**等高级特征。

微调：

调用预先训练好的神经网络，使用更小的训练集进行**二次训练。**

在训练过程中提取训练集特征，对预训练的模型参数经行**微小调整。**

好处：

好处	说明
节省计算资源与时间	无需从随机初始化开始，跳过基础特征提取，训练更快。
小数据集也能训练好模型	缓解数据不足导致的过拟合，提高拟合准确率。
迁移学习能力	通用特征可迁移到不同任务（分类、检测、分割等）
拟合效果更优秀	随机初始化容易陷入较差局部最优

冻结参数：

固定预训练模型中的一些层的权重，即计算梯度反向传播时**不再更新。**

一般冻结卷积层（保留模型的特征提取器）。

作用：

防止过拟合：

数据集小时，全部参数参与更新，模型容易记住训练集噪声。

提高训练效率，减小计算量：

需要计算梯度、更新的参数减少。

保留通用特征：

增强模型的拟合能力。

超参数设置：

学习率大幅度减小，通常用**1e-4 ~ 1e-5。**

学习率分层设置，深层学习率较大，浅层学习率极低甚至0（直接冻结）。

训练轮数适当减少，提高训练效率。

风格迁移：

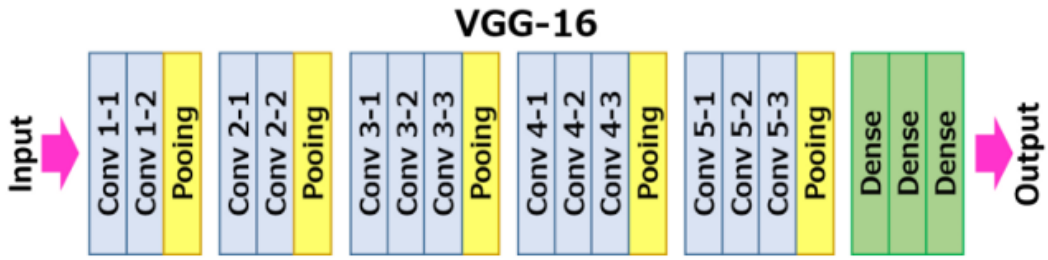
风格迁移中需提取A图特征，B图风格。

提取**内容**：预训练模型可以高效提取A图与生成图的特征值（一般卷积层），以此快速计算差异，即为特征的loss。

提取**风格**：对多层特征图计算Gram矩阵，其可以提取纹理、笔触、颜色分布等风格特征，与空间位置无关。Gram矩阵之差就是风格的loss。

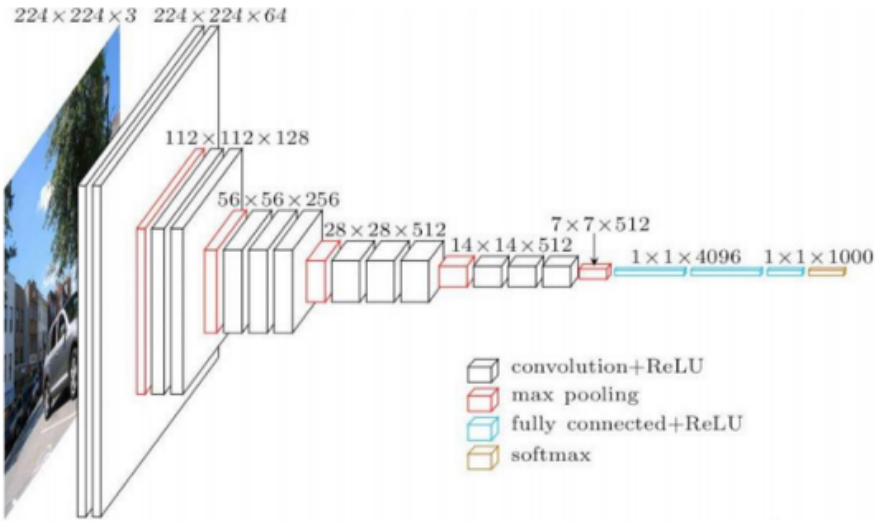
VGG16

VGG16通过13个卷积层与3个全连接层对图像的特征与风格进行提取。



The Architecture

The architecture depicted below is VGG16.



如图VGG16分为6部分，前5部分为**卷积+池化+激活函数**，最后1部分为**全连接。**

卷积：

通过增加**小卷积核数量**来增加图像**通道数。**

同一通道，进过多次卷积来从分提取图像**特征**与**风格。**浅层表示**特征，感受野更小，更注重细节**；深层表示**风格，感受野更大，更注重整体。**

池化：

VGG16使用max-pool来**缩小特征图尺寸，减小参数，扩大感受野，增强平移不变性，提取显著特征。**

激活函数：

引入**非线性**，增强模型拟合能力，卷积、全连接本质上都是**线性变换**，线性变换的叠加并不能增强拟合能力。

全连接：

通过矩阵乘法来**缩小特征图尺寸**，从整体上来提取图像**风格。**

代码细节：

1：

会影响，尺寸差距过大会导致生成图局部模糊或变形。

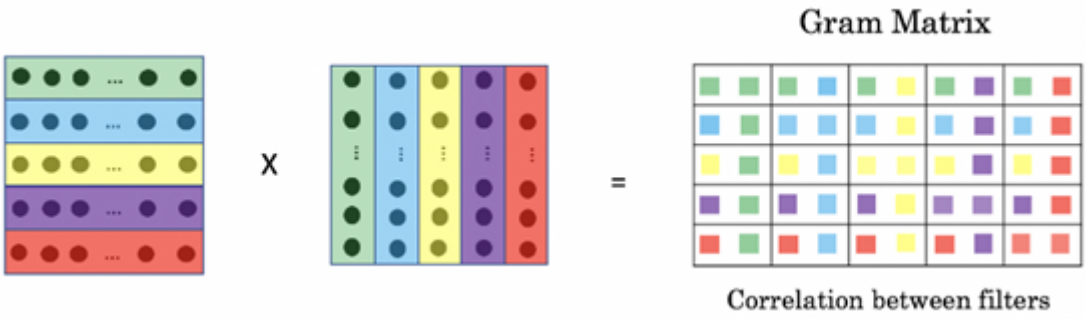
loss：

gram矩阵：

VGG16的特征图是 `(C, H, W)`，C 是通道数，H是高度，W是宽度。

```
c = x.shape[1]
    f = x.reshape(c, -1)
    return f @ f.t()
```

把 `(C, H, W)` 拉平成 `(C, N)`，即把C个高H宽W的二维向量reshape成C行长H × W的一维向量，而其中每个元素代表一个特征。



我们将矩阵 `(C, N)` 转置并与自身进行内积，于是G中每个元素表示多个特征的组合值，因为是不同通道两两相计算使得不同特征得到了组合。

风格损失：

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l - A_{ij}^l\right)^2$$

```
S = gram_matrix(style)
    C = gram_matrix(combination)
    channels = 3
    size = image_height * image_width
    return torch.sum((S - C) ** 2) / (4.0 * channels ** 2 * size ** 2)
```

N = H × W，M为通道数，G_{ij}与A_{ij}分别表示风格图与生成图在VGG中的特征图的gram矩阵，将逐个元素平方差求和。

内容损失：

```
torch.sum((base - combination) ** 2)
```

将内容图与生成图在VGG中的特征图，**逐元素平方差求和**即可。

全变分损失：

```
h, w = x.shape[2], x.shape[3]
a = (x[:, :, : h - 1, : w - 1] - x[:, :, 1:, : w - 1]) ** 2
b = (x[:, :, : h - 1, : w - 1] - x[:, :, : h - 1, 1:]) ** 2
return torch.sum((a + b) ** 1.25)
```

将相邻像素在各方向平方差求和。它可以抑制噪点、让画面更平滑。

4：

因为内容损失中对特征的**位置信息**要求很高，而风格损失则不看重特征的**位置信息**，gram在组合特征的过程中会损失位置信息，正好符合风格损失。

5：

内容图和风格图对应的目标特征是我们生成图的参考参数，是全程不变的。

6：

不是。

冻结参数针对是反向传播，让 VGG 的卷积核不产生梯度、不被优化器更新，保证模型参数不被改动。

评估模式针对是前向传播，它使Dropout与BatchNorm失去作用。

7：

风格**浅层**：感受野小，注重笔画、边缘、颜色斑点 （**纹理**）

内容**深层**：感受野大、 捕捉物体、布局（**内容**）

9：

```
for i in range(iterations + 1):
    optimizer.zero_grad()
    loss = compute_loss(combination_image, base_image, style_image)
    loss.backward()
    optimizer.step()
    scheduler.step()
```

先清空梯度，计算损失（生成图与内容，风格图的差异），再反向计算梯度，更新生成图，再降低学习率。

10：

前向

组合图、内容图、风格图拼成的一个图。

提取其在VGG16相应层下的特征图。

用这些特征算出三种损失按比例相加。

反向

`loss.backward()` 反向计算梯度。

`optimizer.step()` 更新生成图。

`scheduler.step()` 降低学习率。

11：

```
total_variation_weight = 1e-7
style_weight = 5e-4
content_weight = 2.5e-8
image_height = 400
iterations = 4000

optimizer = torch.optim.SGD([combination_image], lr = 100.0)
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma = 0.96 **
(1.0 / 100))
```

平滑程度，风格，内容，图片大小，生成轮数。

由于我生成图直接初始化为内容图，所以`content_weight`极小，而`style_weight`较大。

`total_variation_weight`较小，过大会导致画面模糊。

其实超参数适当即可。

12：

不一定。

比如`total_variation_weight = 0`时，虽然loss减小，但同时画面上会出现大量噪点，导致效果变差。

而且在loss相同的情况下，3种损失函数的比例也会严重影响画面的效果。

调差过程：

先上网查询通用参数，接着生成了大量风格迁移图像，在根据画面效果具体调参。

选做：

1.

风格太重可以适当减小`style_weight`，也可以适当太大`content_weight`。

三种不同风格强度生成结果已在image文件夹中。

2.

更改`style_loss`函数，读入两张风格图，并分别与生成图计算loss，并按一定比例相加作为总的`style_loss`。

```
def S_loss(style1, style2, combination):
    s1 = style1_weight * style_loss(style1, combination)
    s2 = style2_weight * style_loss(style2, combination)
    return s1 + s2
```

其余部分有适当修改，修改后代码在文件夹中。

注：由于两种风格差异巨大，导致loss极大，所以需更改超参数和拉长学习率衰减周期。